

ATMC: Self-Evolving Coding Assistant

Mid-Semester Review Document

Sep 23, 2026 · @Someone

1. The Problem, In Plain Language

AI agents that remember their past experiences to get better over time face a hidden danger: they can accidentally trust their own mistakes.

Think of a student who keeps a notebook of solved problems to study from, but never marks which answers were actually checked and correct. Over time, the notebook fills up with a mix of good and bad advice, all treated equally. Worse, the notebook never gets thrown away or cleaned up — it just keeps growing, making it slower and more expensive to search through, and easier for a single wrong note to quietly become "fact."

This creates two real problems for AI agents that operate over long periods of time:

- **Hallucination propagation** — an incorrect reflection or experience becomes permanent, trusted knowledge, and the agent keeps repeating the same mistake.
- **Context explosion** — memory grows without bound, increasing cost and slowing the agent down, with no way to tell reliable memories from unreliable ones.

This is not a hypothetical concern. It is an active, current research problem — we found a paper published earlier this year attacking a version of this exact issue, which confirms this is a live area of AI research, not an invented exercise.

2. What We Are Building — The Final Vision

The goal: a self-evolving AI coding assistant — an agent that helps write and fix code, and genuinely gets more reliable the longer it works within a codebase, by learning to trust its own good experiences and discard its bad ones.

Concretely, picture an AI assistant working through a sequence of coding tasks in one project. As it goes, it remembers past bugs it fixed and how it fixed them. A naive assistant would treat every past fix as equally trustworthy — including the ones that

were actually wrong — and could keep reapplying a bad fix pattern indefinitely. Our assistant instead tracks which past fixes actually worked, gives more weight to reliable ones, and lets unreliable ones fade away.

Why this is a meaningful real-world target: AI coding assistants are one of the fastest-growing and most heavily used categories of AI tools today. The specific failure mode we are addressing — an assistant confidently repeating its own past mistake — is a genuine, recognized risk in this space, not a made-up scenario.

The core engine behind this assistant is ATMC (Adaptive Trust-based Memory Consolidation) — already built and validated (see Sections 6 and 7). The coding assistant is ATMC put to work in a concrete, real-world setting, wrapped with the additional capabilities (planning, code execution, persistence) that turn a memory mechanism into a full working agent.

3. Research Novelty — Why This Is Publishable

We checked this honestly against existing work before claiming anything is new.

Most of the individual ingredients already exist elsewhere:

- Relevance + recency + importance-based memory ranking — *Generative Agents* (Stanford, 2023)
- Decay/forgetting curves for memory retention — *MemoryBank* (2023)
- A verification step before storing a memory — *Voyager* (2023)
- A paper literally named "TrustMem" (arXiv 2606.25161, 2026) trains a reinforcement-learning model to judge whether a memory *edit* is trustworthy — a related but different problem (memory-editing fidelity, not outcome-based trust)

What is genuinely different about our approach:

1. **Training-free.** The competing TrustMem paper trains an entire model to judge memory quality. Ours is a lightweight formula that works immediately on any off-the-shelf language model — no training pipeline, no extra infrastructure.
2. **Outcome-verified trust.** A memory's trust score is earned by whether relying on it actually led to a correct answer — checked against real ground truth — not by a self-rated importance guess.
3. **A continuously evolving score, not a one-time decision.** Memories keep getting re-evaluated and can lose trust over time, rather than being permanently accepted or rejected the moment they are written.

Honest framing: this is a defensible, incremental research contribution — "a cheaper, training-free way to capture a real, measured share of the benefit that heavier systems provide" — backed by an actual reproducible experimental result, not just a plausible-sounding design.

Because of the name collision with the 2026 TrustMem paper, this project was renamed from its original working name ("TrustMem-Agent") to **ATMC**.

4. Methodology — The Core Mechanism

Every memory the agent stores is scored on four dimensions instead of being treated equally:

Trust (T) — how reliable has this memory proven to be? Updated every time it is used:

- If using it led to a correct outcome, trust goes up a little.
- If it led to a wrong outcome, trust drops by more than it would have gained — bad memories lose trust faster than good memories earn it.

Importance (I) — how relevant is this memory to the current problem, combined with how often it has been usefully retrieved before.

Decay / Forgetting (F) — how much has this memory faded. Untrustworthy or unimportant memories fade fast; reliable, useful ones are kept much longer.

Composite Score — Trust, Importance, and Decay are combined into one ranking score that decides what the agent actually gets to see for a given question. Trust also directly gates Importance: a barely-trusted memory cannot rank highly just by looking superficially relevant — this specific refinement was added after we observed a poisoned-but-highly-similar memory slipping past the ranking in early testing.

Verification Gate — before a new memory is even allowed into the memory bank, it must clear a minimum trust bar. Unverified or clearly-wrong reflections are blocked from ever becoming "permanent knowledge."

Memory Compression — periodically, old and rarely-used memories are merged into a single shorter summary, so the memory bank does not grow forever — this reduces storage and token cost, but our own experiments also uncovered a real limitation: compression can occasionally merge an untrustworthy memory in with clean ones, diluting how obviously unreliable it looks (see Section 7).

Checking correctness: for the tasks tested so far, checking is deterministic — comparing the agent's answer to a known correct answer (math) or running the code against test cases (planned for the coding assistant). This keeps the trust signal objective rather than a subjective guess.

5. System Architecture — The Full Picture

A self-evolving agent needs eight parts, working as a repeating loop:

#	Part	Role
1	Goal	Receive or generate a task to work on
2	Planner	Break the goal into concrete steps
3	Actor	Actually do something — run code, use a tool
4	Checker	Verify whether the attempt succeeded
5	Memory	Decide what to keep, trust, and forget — this is ATMC
6	Reflection	Turn outcomes into reusable insight
7	Curriculum	Choose what to attempt next, autonomously
8	Persistence	Keep accumulating state indefinitely, across sessions

The loop: **Goal → Plan → Act → Check → Update Memory → Reflect → pick next goal → repeat**. Each cycle's output becomes the next cycle's input — this accumulation, not any single part, is what makes the system "self-evolving."

Where we are on this list, honestly:

- **Part A (built and validated): Part 5, the Memory.** ATMC is fully implemented and tested — the hard verification gate, trust-weighted retrieval, adaptive decay, and memory compression. This is the piece proven in Sections 6 and 7 below.
- **Part B (roadmap, not yet built): Parts 1–4 and 6–8.** The Planner, the Actor (code execution), a general Checker, Reflection generation, the Curriculum, and Persistence are the next three months of work (Section 10).

We deliberately built and rigorously validated the hardest, most overlooked piece — reliable memory — before building the rest of the loop around it. This is a considered research strategy, not a shortcut: a self-evolving agent with an unreliable memory is not worth building at all, so proving the memory mechanism works came first.

6. What Has Been Built So Far

The ATMC engine — fully working:

- Core scoring math: Trust, Importance, Decay, Composite Score
- Memory store with the hard verification gate, trust-gated retrieval, and citation-conditioned trust updates (a memory's trust only changes if the AI says it actually relied on it — this closes a real bug we found and fixed mid-project, described below)
- Memory compression, fully implemented and independently switchable on/off
- A deterministic checker (compares answers to known-correct ground truth) and a reasoning agent (Google Gemini)

Experiments that actually ran, not just designed:

- A poisoned-memory experiment: deliberately planted wrong "memories," then measured how often each system let them influence real answers
- Repeated across 3 independent random seeds for statistical reliability
- A full 4-way ablation study isolating exactly which mechanism (Trust / Forgetting / Compression) contributes what

A working full-stack demo application, so the mechanism can be watched live instead of only read from a log file:

- Backend: FastAPI, wrapping the ATMC engine over an API
- Frontend: React — live side-by-side comparison, trust-score visualization, results dashboard

Real engineering process, not a one-shot build. Along the way we found and fixed two genuine flaws in our own method:

1. **Credit-assignment gap** — an early version rewarded a memory's trust just because the overall task succeeded, even if that specific memory was never actually used. Fixed by having the AI self-report which memories it relied on, and only updating trust for those.

2. **Trust-gating gap** — a highly similar (but untrustworthy) memory could rank highly by relevance alone, bypassing the trust check. Fixed by having trust directly discount the relevance score, not just add to it.

Both fixes were empirically re-tested afterward, not just reasoned about — each one measurably improved the result (see Section 7).

7. Key Results

Headline result: 8.0x fewer bad memories reach the AI's reasoning

We seeded two identical memory systems with the same deliberately-wrong "memories," then had both solve the same sequence of real math problems (GSM8K benchmark).

Measured across 3 independent random seeds:

	Baseline (remembers everything)	ATMC (trust-gated)
Accuracy	0.983 ± 0.014	1.000 ± 0.000
Exposure to planted-wrong memories	64.0 ± 4.4	8.0 ± 0.0

8.0x ± 0.54 reduction in exposure to known-bad memories, with zero accuracy cost.

ATMC's exposure count was identical (exactly 8) on every single seed — a deterministic, not lucky, result.

The 4-way ablation study

Isolating exactly which mechanism contributes what (single seed so far; more seeds planned):

Configuration	Accuracy	Poison Exposure	Memory Size	Compressions	Poison "Laundered"
Full System	1.000	9	42	6	2
w/o Trust	1.000	22	42	6	2
w/o Forgetting	1.000	44	42	6	1
w/o Compression	1.000	8	54	0	0

Finding 1 — Forgetting matters more than Trust alone. Removing Trust scoring roughly doubles poison exposure (9→22). Removing Forgetting is worse — nearly 5x (9→44), higher than removing Trust entirely. Trust filtering alone still lets a bad memory sit in the bank forever once admitted; only active forgetting removes it as time passes. This is a genuine, mechanistically-explained insight, not just a confirmation that “the system works.”

Finding 2 — Compression has a real, honest cost. Compression reduces memory size by about 22% (42 vs 54 items) but very slightly increased poison exposure (9 vs 8) — because merging old memories together twice folded a poisoned memory in with clean ones, diluting how distinct it looked (“Poison Laundered” = 2). We are reporting this limitation directly rather than hiding it: compression trades a small safety cost for a real efficiency gain.

8. Tech Stack

Layer	Technology	Purpose
Core engine	Python	Trust/Importance/Decay math, memory store, experiment scripts
Reasoning model	Google Gemini API	The LLM actually solving tasks
Similarity search	scikit-learn (TF-IDF)	Measures relevance between a query and stored memories
Backend	FastAPI	Exposes the engine over a REST API
Frontend	React (Vite)	Live demo: side-by-side comparison, trust visualization, results dashboard
Benchmarks	GSM8K (in use); HumanEval / MBPP (planned)	Math reasoning now; code-execution tasks for the coding-assistant phase
Version control	Git / GitHub	Full commit history, including the bug-fix iterations

A deliberate design choice: **no model training is used anywhere in the currently-built system.** The reasoning model’s weights never change — what evolves is the external memory state (trust scores, stored reflections) that shapes future behavior. This is what

makes the training-free positioning in Section 3 an accurate claim, not just a slogan.

9. Deliverables (End of Semester)

1. **A working self-evolving coding assistant agent** — all 8 architectural parts implemented, operating over real coding tasks with persistent memory across sessions.
2. **A complete experimental results package**: the full 4-way ablation study at 5+ seeds, benchmark results across GSM8K and HumanEval/MBPP, and a baseline comparison (against at least one prior-art-style memory approach).
3. **A research paper draft**, positioned against prior work (Generative Agents, MemoryBank, Voyager, the competing TrustMem paper), with the honest limitations (credit assignment, compression trade-off) included as findings, not hidden.
4. **A polished demo application** showing the coding assistant live — the current math-reasoning demo, extended to the coding-assistant use case.
5. **A public GitHub repository** with full commit history, reproducible experiment scripts, and setup instructions.

Already delivered, ahead of the full timeline: items 1 (partial — the Memory component only) and parts of 2 and 4 above, described in Sections 6–7.

10. Roadmap — Three-Month Plan

Month	Focus	Key additions
1	Harden the core	Full ablation matrix at 5+ seeds, a real Planner agent, self-generated reflections (not just pass/fail tags)
2	Give the agent something to act in	Coding-assistant domain: HumanEval/MBPP, code-execution checker, sandboxed tool execution, persistent database storage
3	Autonomy	Self-directed curriculum (agent chooses its own next task), a reusable skill library, final full evaluation, paper write-up

Each month ends with something concretely stronger than the last — this is a genuinely staged plan, not a padded one. The mid-semester point (now) sits at the end of the core-hardening phase, with the validated Memory mechanism as proof the foundation is solid before the remaining, larger build begins.

11. Summary for the Review

What we set out to do: build a self-evolving AI agent — specifically, a coding assistant — that gets more reliable over time instead of accumulating its own mistakes.

What we proved first, deliberately: the hardest and most overlooked part of that system — a memory mechanism that can tell trustworthy experience from untrustworthy experience — actually works. Not just in theory: an 8.0x reduction in exposure to bad memories, reproduced across 3 seeds, with a 4-way ablation study explaining exactly which mechanism contributes what, including honest limitations we found and are addressing.

What is left: wrapping this validated core in the remaining six parts of the architecture — planning, action, checking, reflection, autonomy, and persistence — aimed specifically at a real-world coding-assistant use case, over the next three months.

Why this approach: proof before scope. We would rather show one rigorously validated mechanism than a fragile end-to-end system with no real evidence behind any of its parts.